프롤로그

환경 차이에 부딪히다

입사 3개월 차 오픈이는 기능 하나를 손보고 있었습니다. 로컬 PC에서는 테스트가 모두 통과했고, 남은 건 운영 서버에 올리는 일뿐이었습니다.

빌드한 파일을 서버에 올리고 실행하자, 터미널이 한 박자 멈춘 뒤 빨간 글씨가 화면을 채웠습니다. 자바 버전이 맞지 않는다는 오류였습니다. 버전을 맞추자 이번엔 라이브러리가 의존하는 시스템 파일이 서버에 없었고, 그 파일을 설치하니 설정 파일의 경로가 서버의 디렉터리 구조와 어긋났습니다.

'내 PC에선 분명히 됐는데.'

하나를 해결하면 곧바로 또 다른 오류가 터졌습니다. 코드는 그대로인데 서버의 환경만 조금씩 어긋나 있었습니다. 한참을 헤매고 있을 때, 옆자리 선배가 모니터를 잠깐 들여다봤습니다.

선배 : "환경이 달라서 그래요. 도커(Docker) 한번 알아봐요."

선배의 말을 들은 오픈이는 퇴근 후 도커를 검색했습니다. 도커란 실행에 필요한 파일과 설정을 **컨테이너**라는 표준 상자에 담아, **어디서든 똑같이 실행**하게 만드는 기술이었습니다.

환경을 컨테이너에 담다

다음 날 저녁, 오픈이는 커피 한 잔을 들고 노트북 앞에 앉았습니다. 전날 본 영상으로 컨테이너와 도커가 왜 필요한지는 이해했고, 이제 남은 건 직접 손에 익히는 일이었습니다.

도커는 가상 머신과 달리 호스트의 운영체제 커널을 공유합니다. 그래서 무거운 OS 전체를 복제하지 않고도, 컨테이너마다 격리된 실행 공간을 가볍게 확보할 수 있었습니다. 먼저 Docker Hub에서 우분투 이미지를 내려받아 컨테이너로 실행하고, 그 안에서 패키지를 설치하고 웹 서버를 실행해 봤습니다.

그렇게 구성한 컨테이너의 상태도 그대로 **이미지**로 저장할 수 있었습니다. 그 이미지를 Docker Hub에 올려 두면, 누구든 어떤 서버에서든 똑같은 환경을 그대로 내려받아 실행할 수 있었습니다. **이미지 하나로 환경을 통째로 옮길 수 있게** 되면서, 환경 차이로 발생하던 문제가 해결됐습니다.

여러 컨테이너를 한 번에 실행하다

며칠 후, 화요일 오전 열 시. 회의실에 사람이 모였습니다. 팀장이 화이트보드 앞에서 이번에 만들 프로젝트를 설명했습니다. 화면을 그릴 프론트엔드, 요청을 처리할 백엔드, 회원 정보를 보관할 DB가 함께 구성된 서비스였습니다.

팀장 : "환경은 도커로 구성해봐요. 요즘 공부했잖아요."

회의실을 나서는 발걸음이 가볍지 않았습니다. 컨테이너 하나를 실행하는 건 익혔지만, 여러 개를 한꺼번에 관리해 본 적은 없었습니다.

'한 대씩은 실행해봤는데, 여러 대를 함께 구성하는 건 또 다른 이야기인데.'

자리로 돌아온 오픈이가 선배에게 물었습니다.

오픈이 : "프론트엔드·백엔드·DB까지 구성해야 하는데, 어디서부터 손대야 할지 모르겠어요."

선배 : "한꺼번에 다 하려니 막막한 거예요. 하나씩 하면 돼요. 서비스마다 필요한 걸 다 갖춘 환경을 미리 준비해 두고, 외부에서 요청을 받아 알맞은 컨테이너로 전달한 뒤, 마지막에 전체를 하나로 합쳐 한 번에 실행하면 돼요."

오픈이는 하나씩 풀어 갔습니다. 먼저 서비스마다 필요한 환경을 **Dockerfile**에 적어 두니, 누가 빌드해도 같은 이미지가 만들어졌습니다.

그다음 요청이 여러 컨테이너로 나뉘어야 할 때는 앞단에 **NGINX**를 세워, 들어온 요청을 경로에 따라 알맞은 백엔드로 분배했습니다. 여러 백엔드가 같은 세션을 공유해야 할 때는 **Redis**에 세션을 모아 두었습니다.

마지막으로 **Docker Compose** 파일 한 장에 서비스들과 연결 관계를 적고 명령어를 입력하자, 흩어져 있던 컨테이너가 명세서대로 함께 실행되며 **하나의 웹 서비스로 동작**했습니다.

완성한 프로젝트를 회의실에서 시연했습니다. 시연은 문제없이 끝났지만, 팀장이 마지막에 한 가지를 물었습니다.

팀장 : "환경 구성은 깔끔하네요. 그런데 운영 중에 컨테이너 하나라도 죽으면 누가 살리죠?"

오픈이는 그 질문에 답하지 못했습니다.

컨테이너를 자동으로 관리하다

회의실을 나선 뒤에도 그 질문이 머릿속을 떠나지 않았습니다. 도커로 컨테이너를 실행할 수는 있어도, 죽은 컨테이너를 되살리는 일까지는 도커만으로 답이 보이지 않았습니다.

점심을 먹고 돌아오는 길에 선배에게 고민을 털어놓았습니다.

오픈이 : "도커를 공부하면 끝일 줄 알았는데, 운영에서 컨테이너가 죽거나 트래픽이 몰릴 때 어떻게 해야 할지 모르겠어요."

선배 : "도커만으로는 운영까지 감당하기 어려워요. 그래서 쿠버네티스(Kubernetes)를 같이 알아야 해요. 컨테이너 관리는 쿠버네티스 담당이거든요."

쿠버네티스는 명령을 하나하나 내리는 대신, 유지할 상태를 선언해 두면 시스템이 그 상태를 알아서 유지하는 방식이었습니다.

오픈이는 **Deployment** 설정 파일에 유지할 Pod 개수를 적어 배포한 뒤, 고의로 Pod 하나를 종료해 봤습니다. 쿠버네티스는 즉시 빈자리를 감지하고 새 Pod를 생성해 원래 개수를 맞췄습니다.

새 버전을 올릴 때도 한꺼번에 바꾸지 않고 Pod를 하나씩 교체하는 **롤링 업데이트**로, 서비스가 끊기지 않았습니다. 이제 컨테이너가 종료되거나 트래픽이 몰려도, **시스템이 선언한 상태를 스스로 유지**하게 되었습니다.

외부에서 접속할 주소를 만들다

오픈이는 학습한 내용을 바탕으로 테스트 서버를 Pod로 재구성했습니다. Pod가 종료되더라도 Deployment가 즉시 새 Pod를 실행해 주니 마음이 든든했습니다.

다음 날 아침, 동료가 찾아왔습니다.

동료 : "오픈 씨, 어제 테스트 서버에 올렸다는 서비스, 접속이 안 되는데 어떻게 들어가요?"

오픈이는 당황했습니다. 로그상으로는 정상 실행 중이었지만, 정작 브라우저로 접속해 본 적은 없었습니다. Pod 목록에서 IP를 확인해 입력해 봐도 페이지는 열리지 않았고, 게다가 Pod가 재생성될 때마다 IP가 바뀌니 그 주소로는 접근할 수도 없었습니다.

'쿠버네티스 환경에서는 외부에서 Pod로 어떻게 접속해야 하지?'

막막해진 오픈이가 선배에게 묻자, 선배가 화면을 보며 설명했습니다.

선배 : "외부 요청이 실제 Pod까지 도달하려면 두 가지가 필요해요. 들어온 요청을 경로에 따라 알맞은 곳으로 보내 주는 Ingress, 그리고 수시로 바뀌는 Pod에 고정된 진입점을 제공하는 Service죠."

오픈이는 IP가 계속 바뀌는 Pod들 앞에 고정된 주소를 제공하는 **Service**를 배치했습니다. 그다음 **Ingress**를 통해 외부에서 들어온 요청을 경로에 따라 알맞은 Service로 넘겨주었습니다.

구성을 마치자 외부 브라우저의 요청이 Ingress와 Service를 거쳐 Pod 안의 애플리케이션에 정상적으로 도달했습니다. 이제 IP가 계속 바뀌어도, **외부에서 고정된 주소로 서비스에 접속**할 수 있게 되었습니다.

설정과 데이터를 따로 관리하다

며칠 뒤, 오픈이는 프로젝트를 배포하기 전 마지막으로 코드 리뷰를 하고 있었습니다. 그러던 중 Dockerfile의 환경 변수가 눈에 띄었습니다. 데이터베이스 비밀번호가 코드에 그대로 적혀 있었습니다.

'잠깐. 이대로 빌드하면 비밀번호가 이미지에 그대로 남는데.'

오픈이가 옆자리 선배에게 묻자, 선배가 의자를 돌려 오픈이를 봤습니다.

선배 : "테스트 환경에서는 괜찮지만, 실제 운영 환경에 배포할 때는 두 가지를 주의해야 해요. 비밀번호 같은 설정값은 자주 바뀌니 Dockerfile에 직접 적지 말고 이미지 밖으로 분리해서 관리하고, Pod는 언제든 종료될 수 있으니 데이터는 Pod 안이 아니라 바깥 저장소에 둬요. 설정과 데이터 모두 컨테이너 밖으로 안전하게 빼 두는 게 핵심이에요."

오픈이는 환경별 설정값을 **ConfigMap**에, 민감한 비밀번호는 **Secret**에 분리해 Pod 실행 시 외부에서 주입되도록 수정했습니다.

데이터베이스의 실제 데이터는 Pod 내부가 아닌 **PV(Persistent Volume)** 라는 외부 저장 공간을 연결해 보관했습니다. 컨테이너가 삭제되거나 재생성되어도 **설정과 데이터는 외부에서 유지**되었습니다.

프론트엔드, 백엔드, DB가 외부 설정 파일을 참조하며 동작하도록 구성을 마쳤습니다. 도커로 실행 환경을 규격화하고, 쿠버네티스로 운영까지 맡기는 구조가 완성됐습니다.

오픈이는 완성된 시스템을 선배에게 보여 주었습니다.

오픈이 : "환경의 차이를 해결하려고 시작했는데, 이제 시스템이 자동으로 운영되는 것까지 해결됐네요."

선배는 흐뭇한 표정으로 답했습니다.

선배 : "처음부터 모든 걸 알고 시작하는 사람은 없어요. 부딪히고, 배우고, 익히다 보면 되는 거예요."